

Compiladores 2026 — Fase 2

Guía de pruebas en vivo

1. Setup

Tkinter (recomendado para presentar)

```
python "files (1)/main.py"
```

- **F5** = Analizar.
- Botón **Ejemplos** (abajo a la izquierda) carga los 7 programas precargados.

Web (alternativa, si tenés XAMPP/WAMP corriendo)

1. Copiar files (1)/ a htdocs/minicompiler/.
2. Abrir <http://localhost/minicompiler/index.php>.
3. Botón **Analizar** arriba.

2. Pruebas a presentar (orden recomendado)

Test 1 — Pipeline completo: frontend → TAC → backend optimizado

Código:

```
int a = 3;
int b = 4;
int c = a + b;
print(c);
```

Qué mostrar:

- Tab **Tokens**: 16 tokens detectados.
- Tab **Árbol**: AST con nodos Program → Declaration → BinaryOp(+).
- Tab **Código Intermedio** → **Original**: \$t1 = a + b; c = \$t1; print c.
- Tab **Código Intermedio** → **Optimizado**: c = 7; print 7 (reducción ~30 %).

Qué decir: “Acá se ve el flujo completo: el frontend produce el TAC, el backend lo simplifica sin tocar el lenguaje fuente. Esa es la frontera.”

Test 2 — Constant Folding

Código:

```
int x = (3 + 4) * 2 - 5;
print(x);
```

Esperado:

- Original: 5 cuádruplos con \$t1, \$t2, \$t3.
- Optimizado: x = 9; print 9.

Qué decir: “Como los dos operandos son constantes, el optimizador evalúa la expresión en compilación. La aritmética desaparece del código generado.”

Test 3 — Identidades algebraicas

Código:

```
int x = 7;
int y = x + 0;
int z = x * 1;
int w = x * 0;
print(y); print(z); print(w);
```

Esperado:

- y y z se reducen a copias directas de x.
- w se reemplaza por 0 literal.
- Los temporales intermedios desaparecen por dead-code.

Qué decir: “x+0, x*1 y x*0 se reescriben sin necesidad de evaluar — son identidades.”

Test 4 — Branch Pruning

Código:

```
if (true) {
    print(1);
} else {
    print(2);
}
```

Esperado:

- Original: tiene `ifFalse true goto $L1`, ambas ramas, etiquetas.
- Optimizado: solo `print 1`. La rama `else` y los saltos desaparecen.

Qué decir: “Después de copy-propagation la condición queda como literal `true`. La pasada de Branch Pruning detecta el salto inútil y lo elimina; después Jump Threading limpia las etiquetas huérfanas.”

Test 5 — Control de flujo (while)

Código:

```
int n = 0;
int i = 0;
while (i < 3) {
    n = n + i;
    i = i + 1;
}
print(n);
```

Qué mostrar:

- En el TAC se ve la estructura del while: `$L1: ...ifFalse ...goto $L2 ...goto $L1 ...$L2:`.
- Las variables del lazo **no** se foldean (porque su valor cambia en cada iteración).
- El optimizador respeta los bordes de bloque (las etiquetas invalidan el entorno de propagación).

Qué decir: “Acá se ve por qué la propagación se invalida en saltos: si el optimizador asumiera que i sigue valiendo 0 dentro del lazo, rompería el programa.”

Test 6 — For completo

Código:

```
int s = 0;
for (int i = 0; i < 5; i = i + 1) {
    s = s + i;
}
print(s);
```

Qué mostrar: misma idea que el while; se ve cómo **for** se traduce a TAC con etiqueta de inicio, condición, cuerpo, actualización y salto.

Test 7 — Recuperación ante errores

Código:

```
int x = 5@;
int x = 10;
string s = "sin cerrar
```

Qué mostrar:

- Tab **Errores** lista los 3 problemas:
 - Carácter ilegal '@' (léxico)
 - 'x' ya fue declarado (semántico)
 - Carácter ilegal '"' (cadena sin cerrar)
- A pesar de los errores, el TAC **se sigue generando** para la parte válida.

Qué decir: “El compilador no aborta al primer error: reporta todos y sigue generando lo que puede.”

Test 8 — Unary minus (bug recién arreglado)

Código:

```
int x = -5;
int y = -x;
print(y);
```

Esperado:

- Original: \$t1 = 0 - 5; x = \$t1; \$t2 = 0 - x; y = \$t2; print y
- Optimizado: x = -5; y = 5; print 5

Qué decir: “El unario menos se traduce a 0 - operando, y el optimizador lo dobla automáticamente.”

Test 9 — Coloreado del editor (solo Tkinter)

Código:

```
string saludo = "Hola Mundo";
int edad = 25;
boolean activo = true;
float pi = 3.1416;
```

Qué mostrar: después de F5, en el editor:

- `string`, `int`, `boolean`, `float` en celeste (palabras clave).
- "Hola Mundo" en verde — **incluyendo las dos comillas**.
- 25, 3.1416 en naranja.
- `true` en amarillo.

Qué decir (si alguien pregunta): “Antes el highlight se cortaba un par de caracteres antes del final del string; ahora usa el largo del texto fuente, no el del valor procesado.”

Test 10 — Vista visual del optimizador (la pieza más vendible)

Código (programa que produce mucha reducción):

```
int a = 3;
int b = 4;
int c = a + b;
int d = c * 2;
int e = d - 0;
int f = e * 1;
int g = f * 0;
print(g);
```

Qué mostrar en la pestaña Código Intermedio:

1. **Sub-tab Original** — 14+ cuádruplos.
2. **Sub-tab Optimizado** — 1-2 cuádruplos (`print 0`).
3. **Sub-tab Comparación** — los dos lado a lado.
4. **Sub-tab Info** — explicación de las pasadas.
5. **Métricas en el header:** Cuádruplos: 14 → 2 · Reducción: 86 % · Temporales: 4 → 0.
6. **Tabla de traza** abajo: cada pasada que hizo cambios, con `iter` / `pasada` / `antes` / `después` / `delta`.

Qué decir: “Esta vista sirve también como apoyo didáctico: no solo qué se redujo, sino *por qué* pasada y en qué iteración.”

3. Si algo falla en vivo

Síntoma	Causa probable	Salida rápida
Web carga pero TAC sale vacío	PHP no encuentra Python	Caer al modo JS local (deshabilitar backend con devtools) o usar Tkinter
Tkinter no abre el menú de Ejemplos	Foco perdido en otra ventana	Click en el editor primero, después en Ejemplos
python3: command not found (web Linux)	Falta alias	<code>sudo ln -s /usr/bin/python /usr/bin/python3</code>
El árbol no se dibuja (web)	Tokens vacíos por error léxico	Mirar el tab Errores ; corregir el código fuente

4. Tips para la presentación

- **Tené las 10 muestras pegadas en un archivo de texto aparte** — copiar/pegar es más rápido y profesional que tipear en vivo.

- **Empezá por el Test 1** (pipeline completo) y **terminá con el Test 10** (vista visual con mucha reducción) — el efecto *wow* está en ver $14 \rightarrow 2$ cuádruplos.
- Si presenta otra persona y se traba, los 7 ejemplos del menú **Ejemplos** ya están precargados como red de seguridad — el #4 (“Expresiones Complejas”) da $33 \rightarrow 15$ (~ 55 % reducción), buen fallback.
- Antes de la presentación: corré los 7 ejemplos una vez en orden para confirmar que la máquina anda fina.